

Module-1 JavaScript

Project Theme: "Local Community Event Portal"

Users can view upcoming events, register, filter events by category or location, and interact dynamically with the portal.

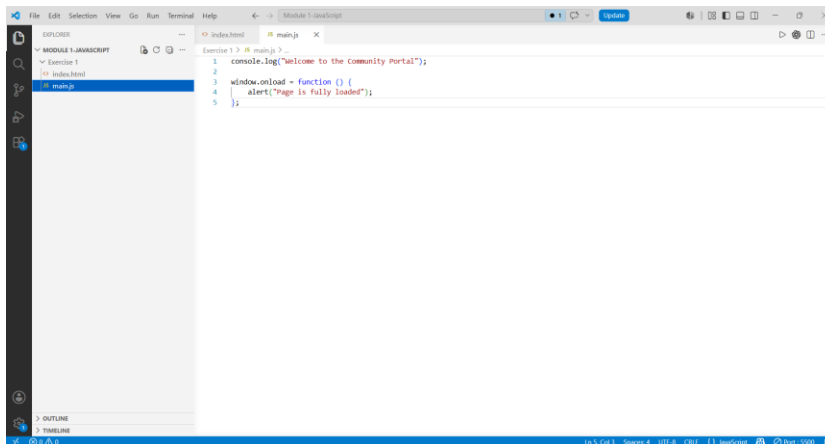
1. JavaScript Basics & Setup

Scenario: Set up your community portal to use JavaScript.

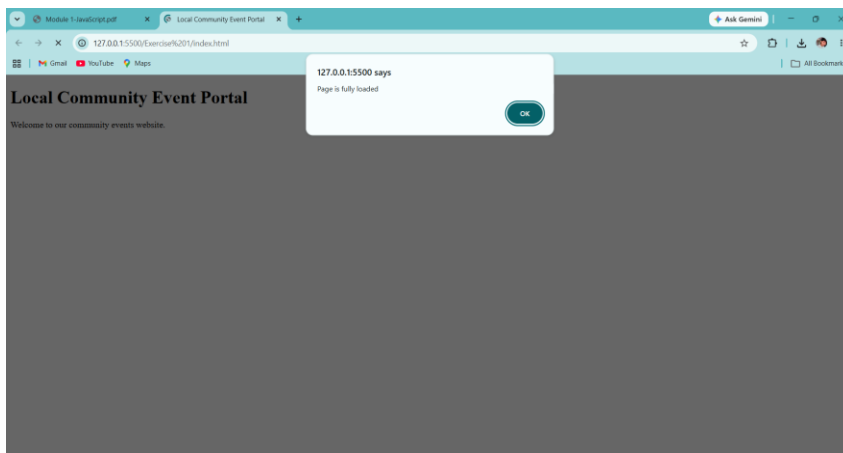
Objective: Configure environment and test basic script functionality.

Task:

- Use in HTML
- Log "Welcome to the Community Portal" using console.log()
- Use an alert to notify when the page is fully loaded



Output:



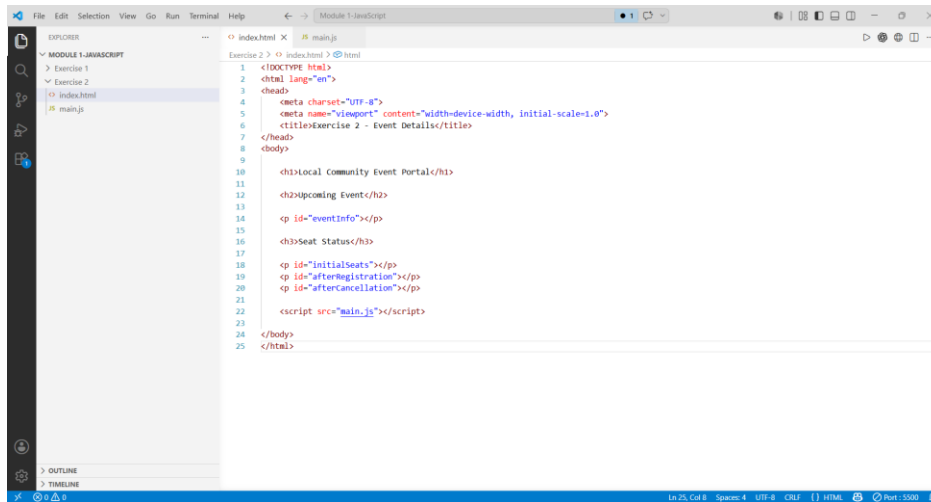
2. Syntax, Data Types, and Operators

Scenario: Store event details like name, date, and available seats.

Objective: Use proper data types and operations.

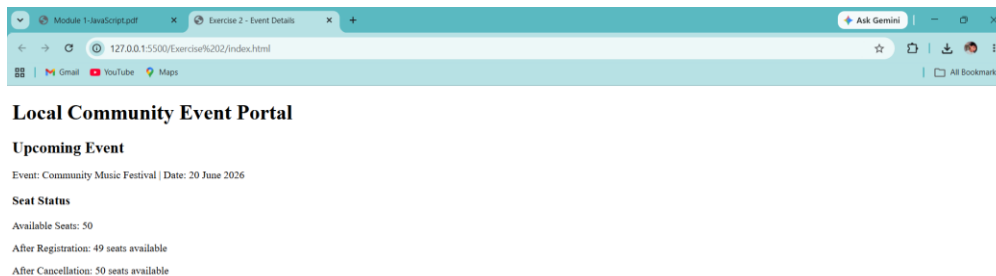
Task:

- Use const for event name and date, let for seats
- Concatenate event info using template literals
- Use ++ or -- to manage seat count on registration



```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Exercise 2 - Event Details</title>
7   </head>
8   <body>
9
10    <h1>Local Community Event Portal</h1>
11
12    <h2>Upcoming Event</h2>
13
14    <p id="eventInfo"></p>
15
16    <h3>Seat Status</h3>
17
18    <p id="initialSeats"></p>
19    <p id="afterRegistration"></p>
20    <p id="afterCancellation"></p>
21
22    <script src="main.js"></script>
23
24  </body>
25 </html>
```

Output:



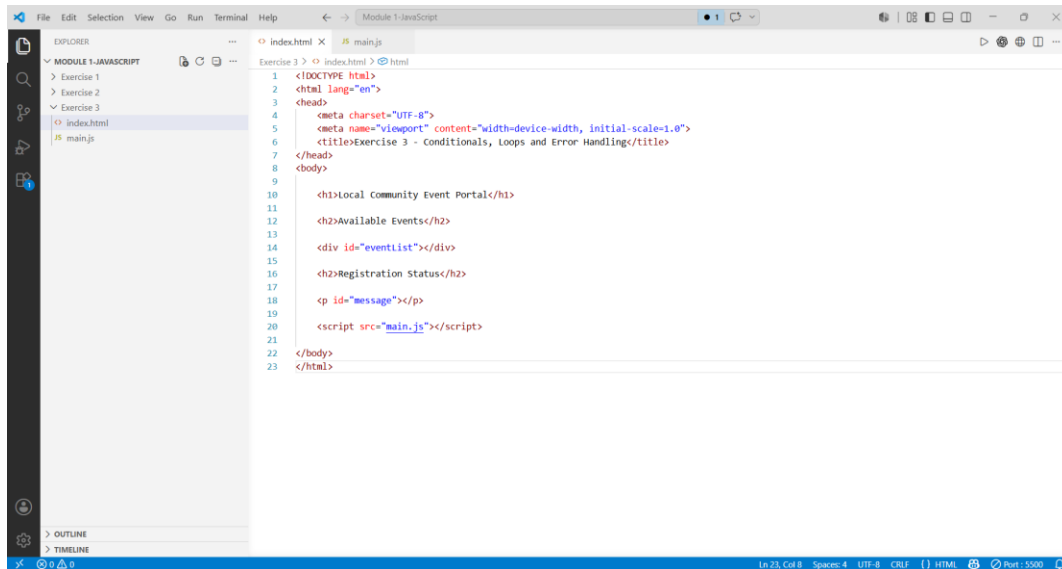
3. Conditionals, Loops, and Error Handling

Scenario: Only show valid events and limit registrations.

Objective: Apply conditions and handle invalid data.

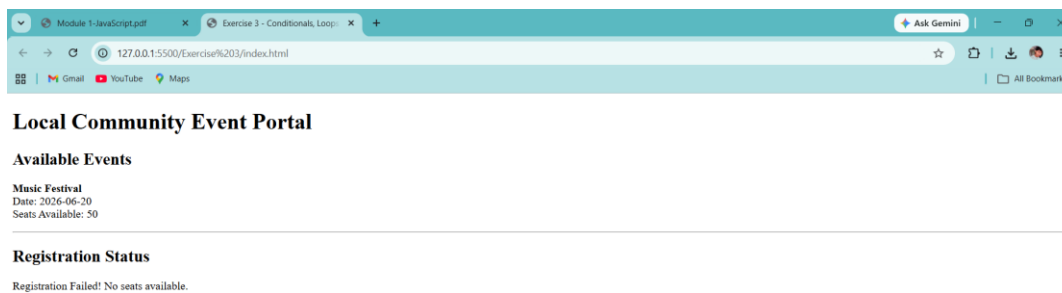
User Story: As a user, I want only upcoming events with seats to be displayed. Task:

- Use if-else to hide past or full events
- Loop through the event list and display using `forEach()`
- Wrap registration logic in try-catch to handle errors



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 3 - Conditionals, Loops and Error Handling</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>Available Events</h2>
13
14  <div id="eventList"></div>
15
16  <h2>Registration Status</h2>
17
18  <p id="message"></p>
19
20  <script src="main.js"></script>
21
22 </body>
23 </html>
```

Output:



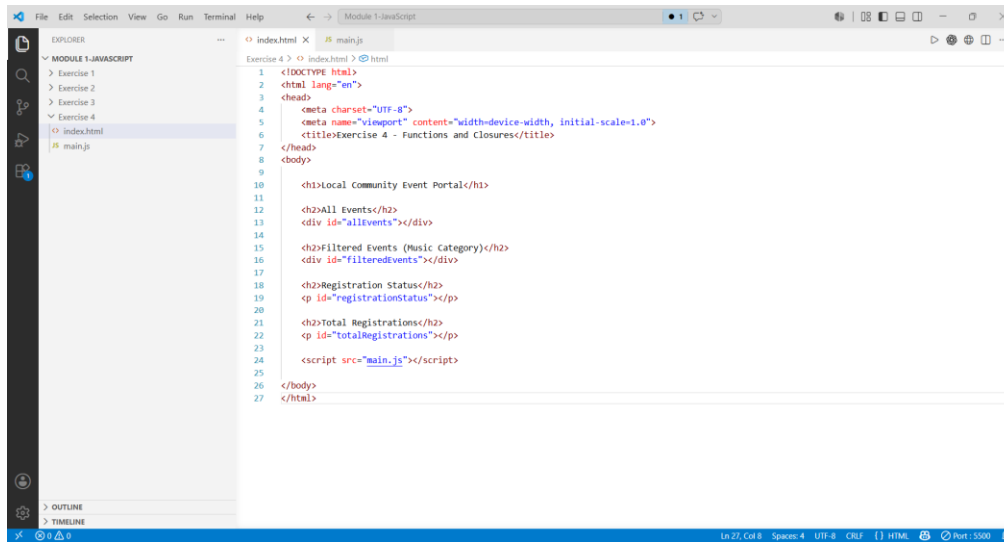
4. Functions, Scope, Closures, Higher-Order Functions

Scenario: Create reusable functions for event operations.

Objective: Encapsulate logic and use closures.

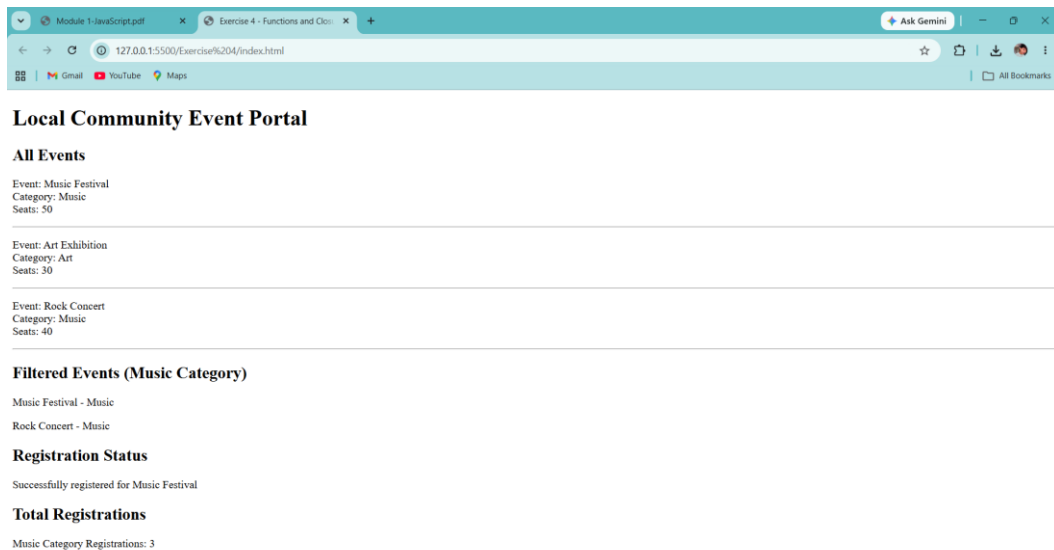
Task:

- Create `addEvent()`, `registerUser()`, `filterEventsByCategory()`
- Use closure to track total registrations for a category
- Pass callbacks to filter functions for dynamic search



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 4 - Functions and Closures</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>All Events</h2>
13  <div id="allEvents"></div>
14
15  <h2>Filtered Events (Music Category)</h2>
16  <div id="filteredEvents"></div>
17
18  <h2>Registration Status</h2>
19  <p id="registrationStatus"></p>
20
21  <h2>Total Registrations</h2>
22  <p id="totalRegistrations"></p>
23
24  <script src="main.js"></script>
25
26 </body>
27 </html>
```

Output:



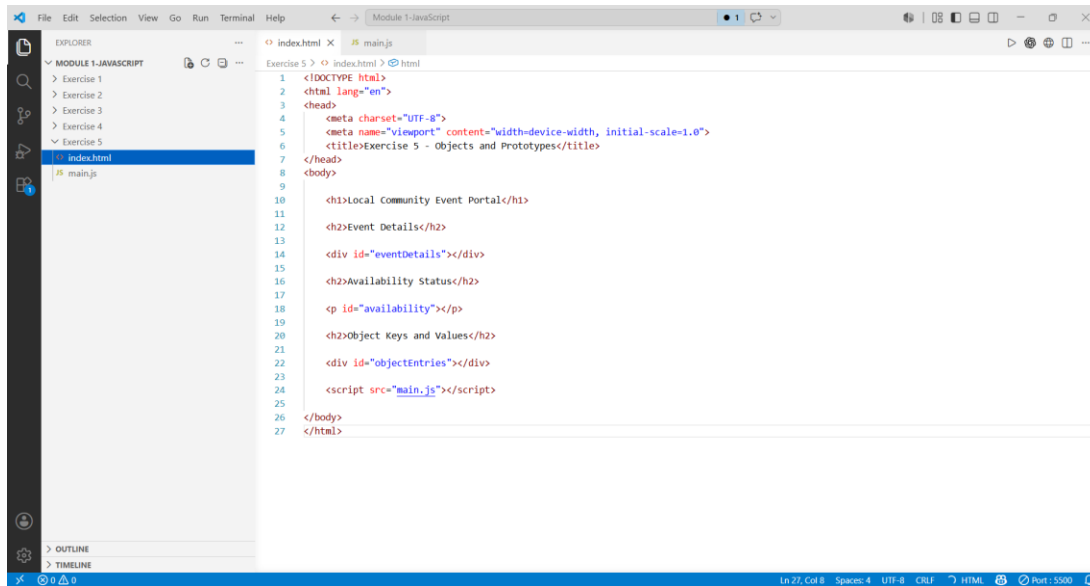
5. Objects and Prototypes

Scenario: Each event is an object with properties and methods.

Objective: Model real-world entities using objects.

Task:

- Define Event constructor or class
- Add checkAvailability() to prototype
- List object keys and values using Object.entries()



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 5 - Objects and Prototypes</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>Event Details</h2>
13
14  <div id="eventDetails"></div>
15
16  <h2>Availability Status</h2>
17
18  <p id="availability"></p>
19
20  <h2>Object Keys and Values</h2>
21
22  <div id="objectEntries"></div>
23
24  <script src="main.js"></script>
25
26 </body>
27 </html>
```

Output:



Local Community Event Portal

Event Details

Event Name: Music Festival

Date: 20 June 2026

Category: Music

Seats: 50

Availability Status

Seats Available

Object Keys and Values

name : Music Festival

date : 20 June 2026

category : Music

seats : 50

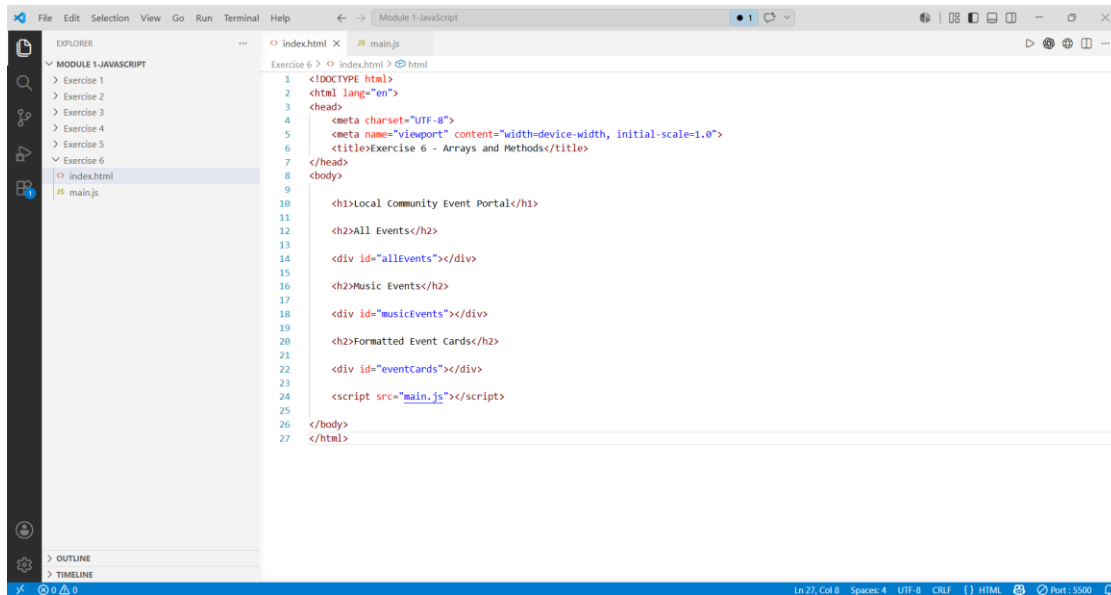
6. Arrays and Methods

Scenario: Manage an array of all community events.

Objective: Use array methods for CRUD operations.

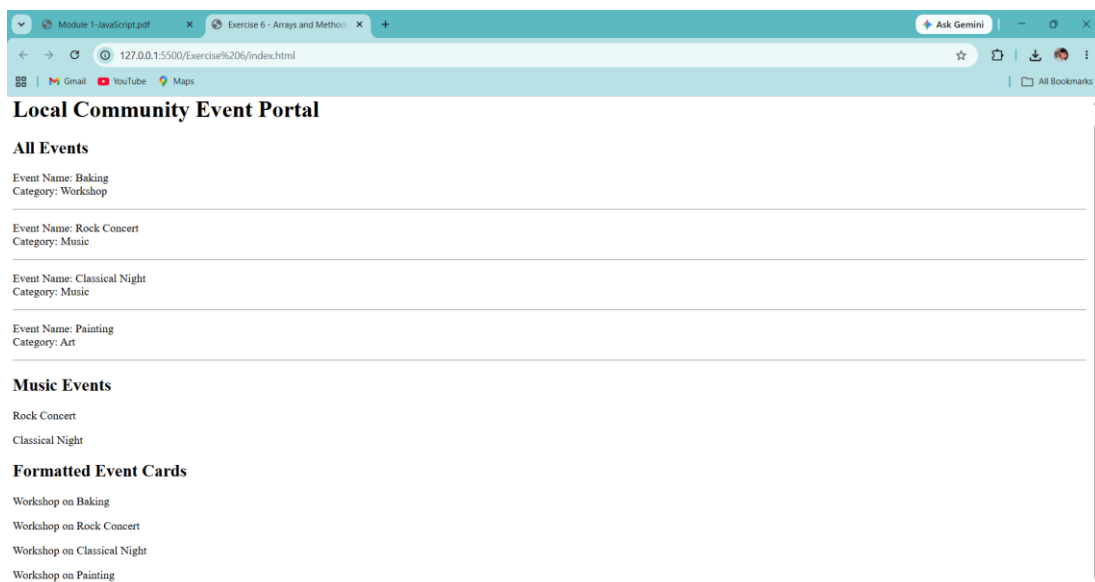
Task:

- Add new events using `.push()`
- Use `.filter()` to show only music events
- Use `.map()` to format display cards (e.g., "Workshop on Baking")



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 6 - Arrays and Methods</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>All Events</h2>
13
14  <div id="allEvents"></div>
15
16  <h2>Music Events</h2>
17
18  <div id="musicEvents"></div>
19
20  <h2>Formatted Event Cards</h2>
21
22  <div id="eventCards"></div>
23
24  <script src="main.js"></script>
25
26 </body>
27 </html>
```

Output:



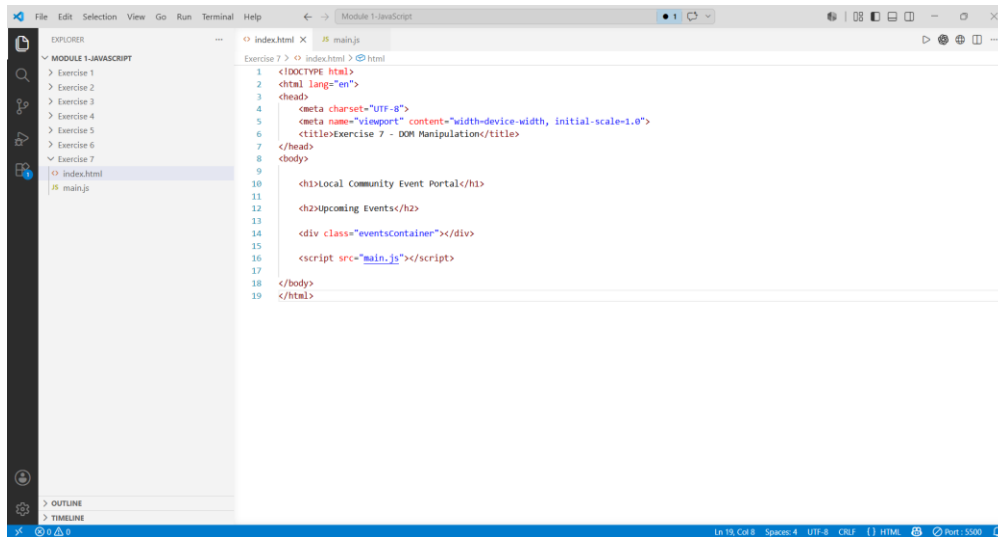
7. DOM Manipulation

Scenario: Display all events dynamically on the webpage.

Objective: Render events using JS

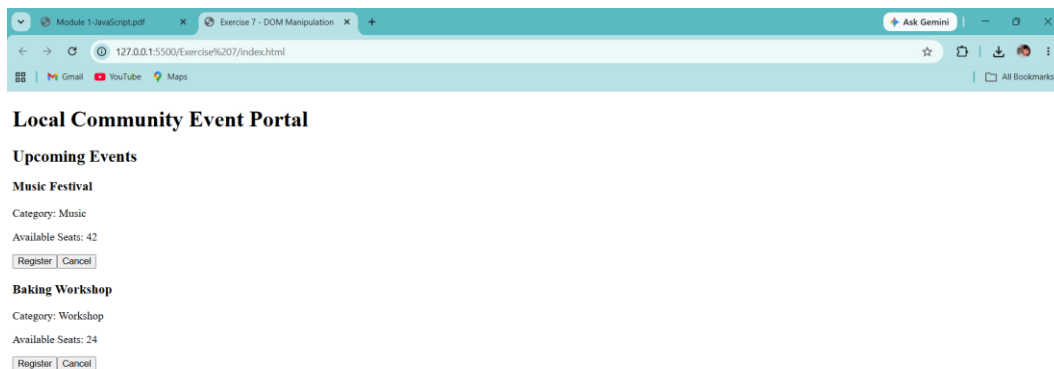
. Task:

- Access DOM elements using `querySelector()`
- Create and append event cards using `createElement()`
- Update UI when user registers or cancels



```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Exercise 7 - DOM Manipulation</title>
7   </head>
8   <body>
9
10    <h1>Local Community Event Portal</h1>
11
12    <h2>Upcoming Events</h2>
13
14    <div class="eventscontainer"></div>
15
16    <script src="main.js"></script>
17
18  </body>
19 </html>
```

Output:



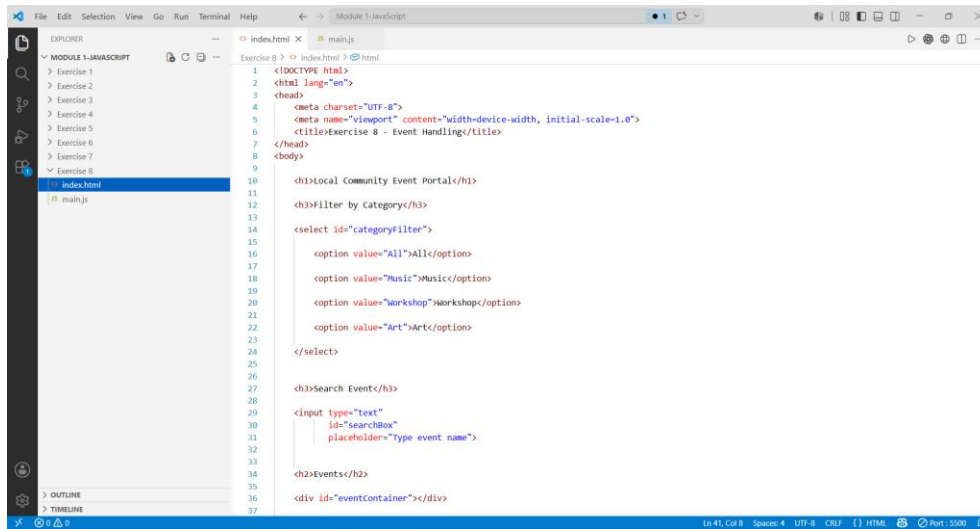
8. Event Handling

Scenario: Add interactive elements like buttons and filters.

Objective: Respond to user actions.

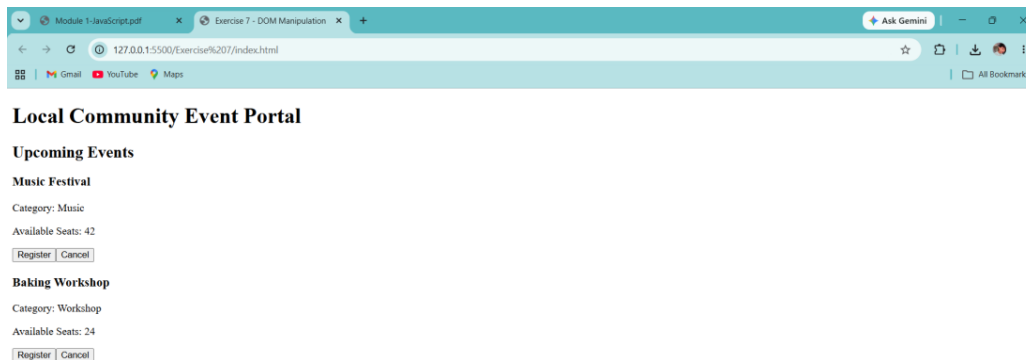
Task:

- Use onclick for "Register" buttons
- Use onchange to filter events by category
- Use keydown to allow quick search by name



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 8 - Event Handling</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h3>Filter by Category</h3>
13
14  <select id="categoryFilter">
15    <option value="All">All</option>
16    <option value="Music">Music</option>
17    <option value="Workshop">Workshop</option>
18    <option value="Art">Art</option>
19  </select>
20
21  <h3>Search Event</h3>
22  <input type="text"
23    id="searchBox"
24    placeholder="Type event name">
25
26  <h2>Events</h2>
27
28  <div id="eventContainer"></div>
```

Output:



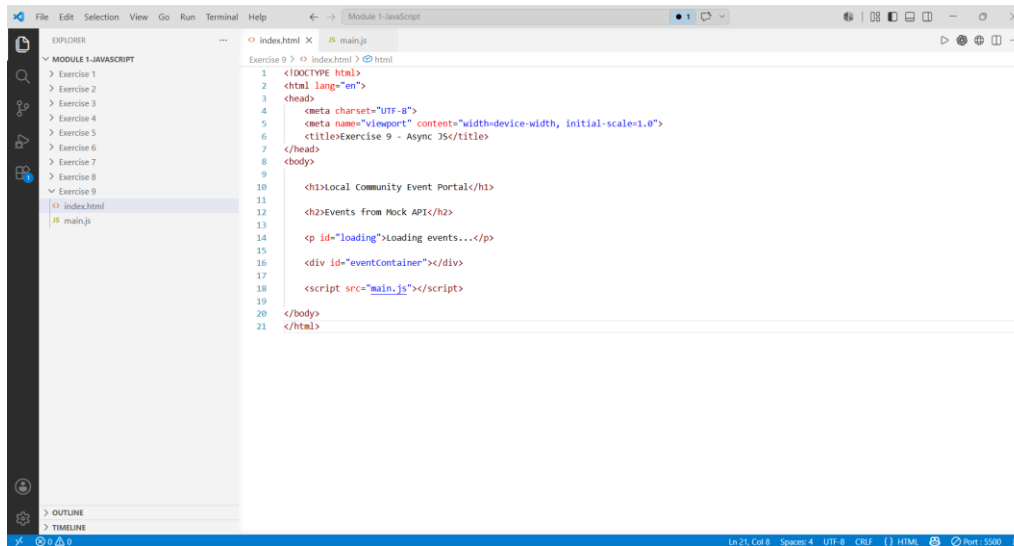
9. Async JS, Promises, Async/Await

Scenario: Fetch event data from a mock API.

Objective: Use asynchronous logic for remote operations.

Task:

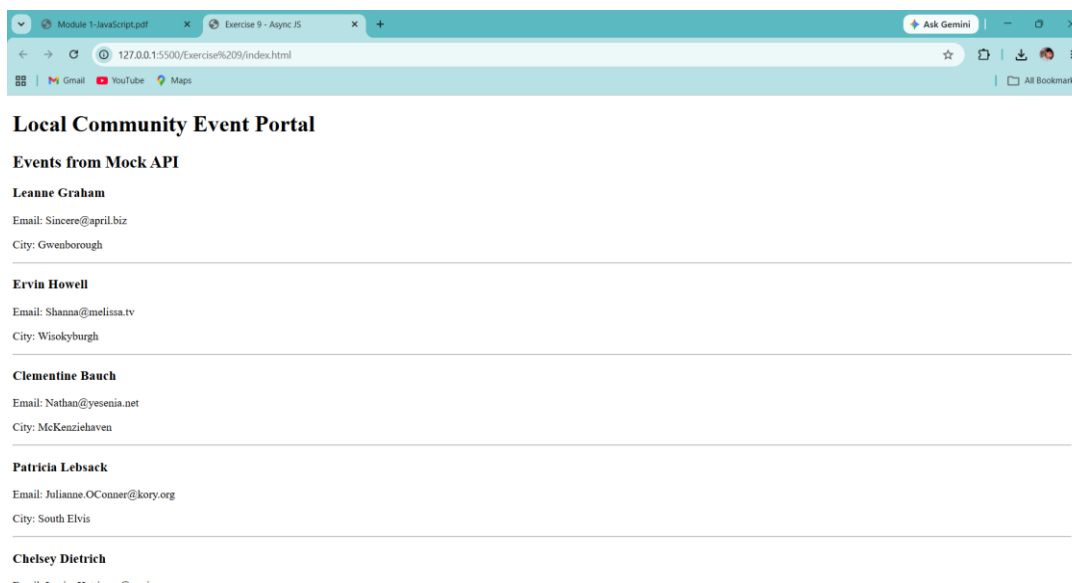
- Fetch events from a mock JSON endpoint
- Use `.then()` and `.catch()` to handle results
- Rewrite using `async/await` and show loading spinner



The screenshot shows the VS Code editor with two files open: `index.html` and `main.js`. The `index.html` file contains the following HTML structure:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 9 - Async JS</title>
7 </head>
8 <body>
9
10 <h1>Local Community Event Portal</h1>
11
12 <h2>Events from Mock API</h2>
13
14 <p id="loading">Loading events...</p>
15
16 <div id="eventContainer"></div>
17
18 <script src="main.js"></script>
19
20 </body>
21 </html>
```

Output:



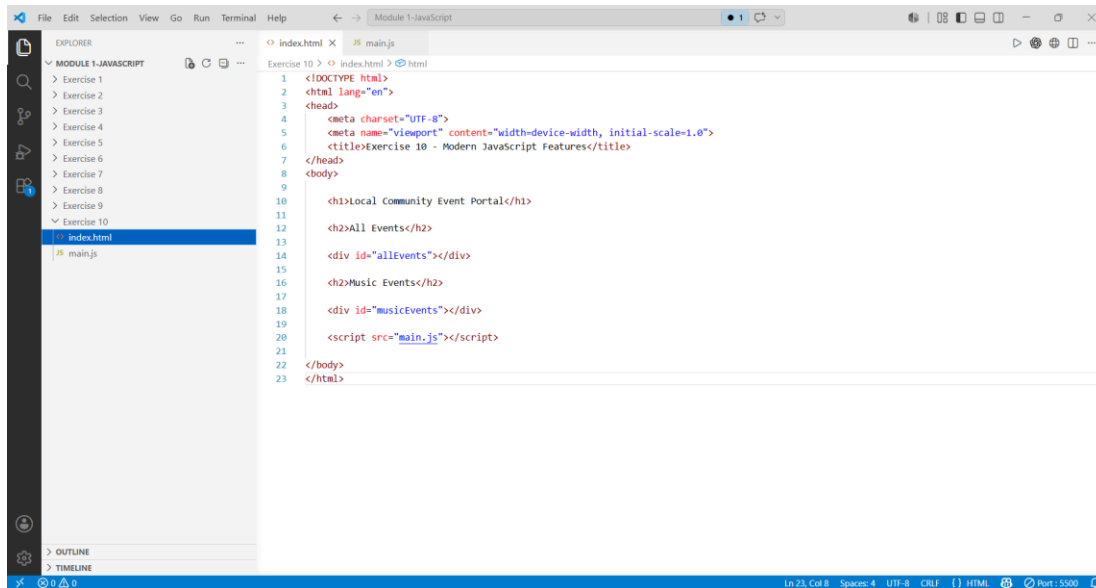
10. Modern JavaScript Features

Scenario: Refactor code to be concise and maintainable.

Objective: Use ES6+ features.

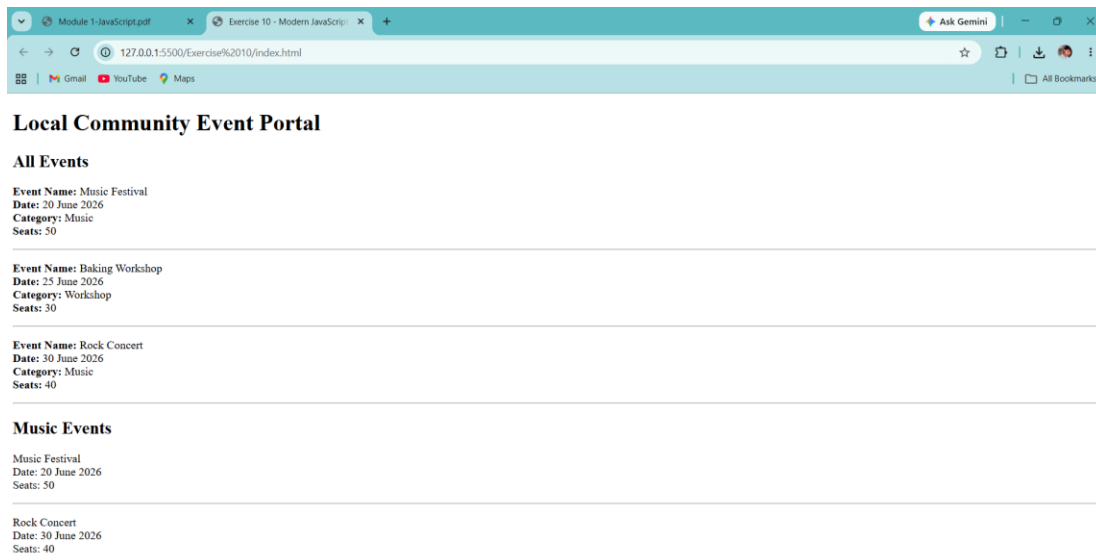
Task:

- Use let, const, default parameters in functions
- Use destructuring to extract event details
- Use spread operator to clone event list before filtering



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 10 - Modern JavaScript Features</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>All Events</h2>
13
14  <div id="allEvents"></div>
15
16  <h2>Music Events</h2>
17
18  <div id="musicEvents"></div>
19
20  <script src="main.js"></script>
21
22 </body>
23 </html>
```

Output:



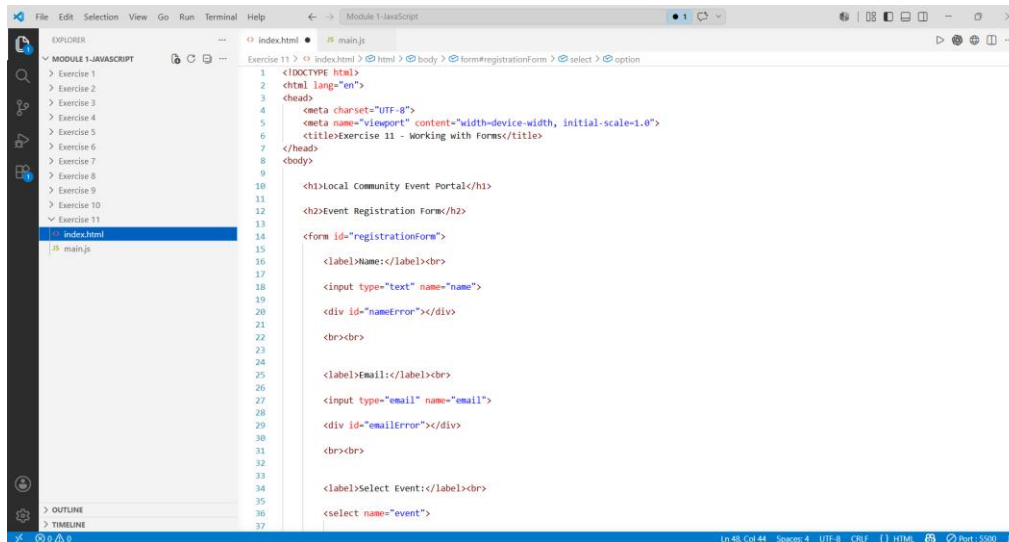
11. Working with Forms

Scenario: Create a registration form for event sign-up.

Objective: Connect form inputs to JavaScript.

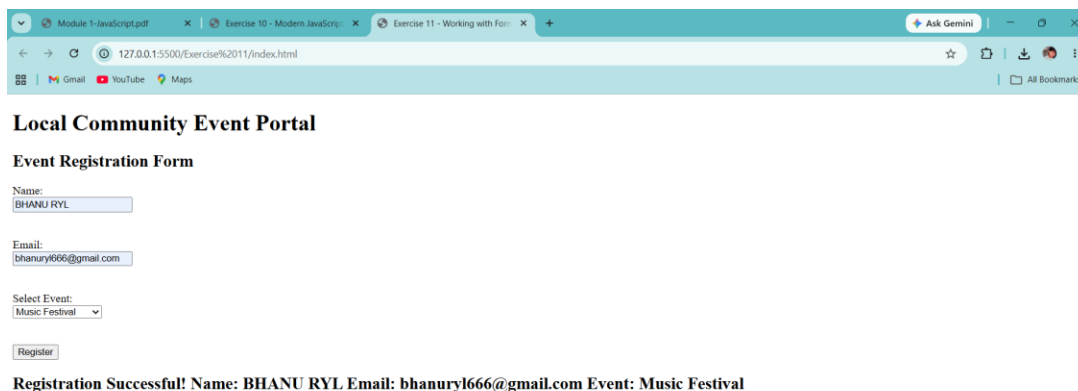
Task:

- Capture name, email, and selected event using form.elements
- Prevent default form behavior using event.preventDefault()
- Validate inputs and show errors inline



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 11 - Working with Forms</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>Event Registration Form</h2>
13
14  <form id="registrationForm">
15
16    <label>Name:</label><br>
17    <input type="text" name="name">
18    <div id="nameError"></div>
19
20    <br><br>
21
22    <label>Email:</label><br>
23    <input type="email" name="email">
24    <div id="emailError"></div>
25
26    <br><br>
27
28    <label>Select Event:</label><br>
29    <select name="event">
30
31  </form>
```

Output:



Local Community Event Portal

Event Registration Form

Name:
BHANU RYL

Email:
bhanury666@gmail.com

Select Event:
Music Festival

Register

Registration Successful! Name: BHANU RYL Email: bhanury666@gmail.com Event: Music Festival

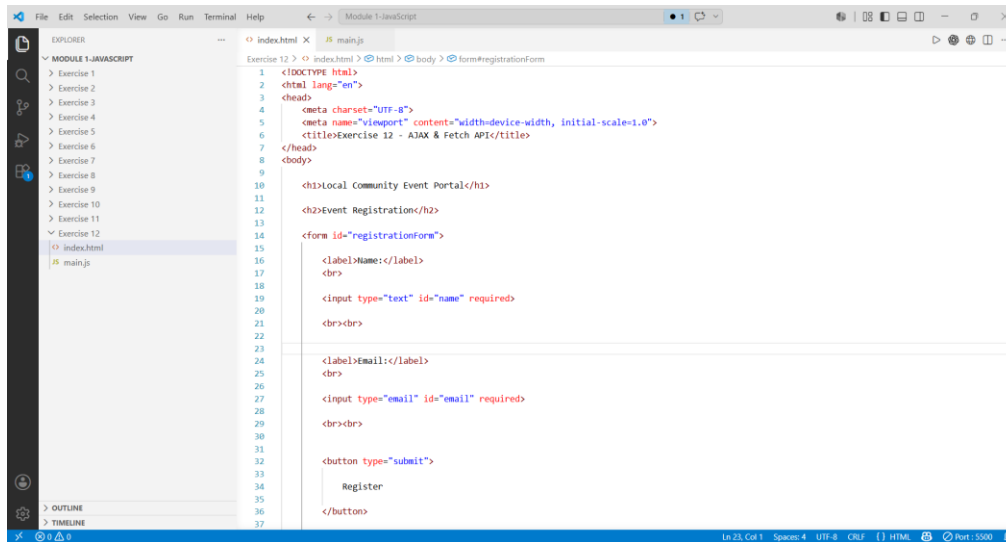
12. AJAX & Fetch API

Scenario: Send user registration to the server.

Objective: Simulate backend communication.

Task:

- Use `fetch()` to POST user data to a mock API
- Show success/failure message after submission
- Use `setTimeout()` to simulate a delayed response



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 12 - AJAX & Fetch API</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portals</h1>
11
12  <h2>Event Registration</h2>
13
14  <form id="registrationForm">
15
16    <label>Name:</label>
17    <br>
18    <input type="text" id="name" required>
19
20    <br><br>
21
22    <label>Email:</label>
23    <br>
24    <input type="email" id="email" required>
25
26    <br><br>
27
28    <button type="submit">
29      Register
30    </button>
31
32  </form>
33
34
35
36
37
```

Output:



Local Community Event Portal

Event Registration

Name:

Email:

Registration Successful!

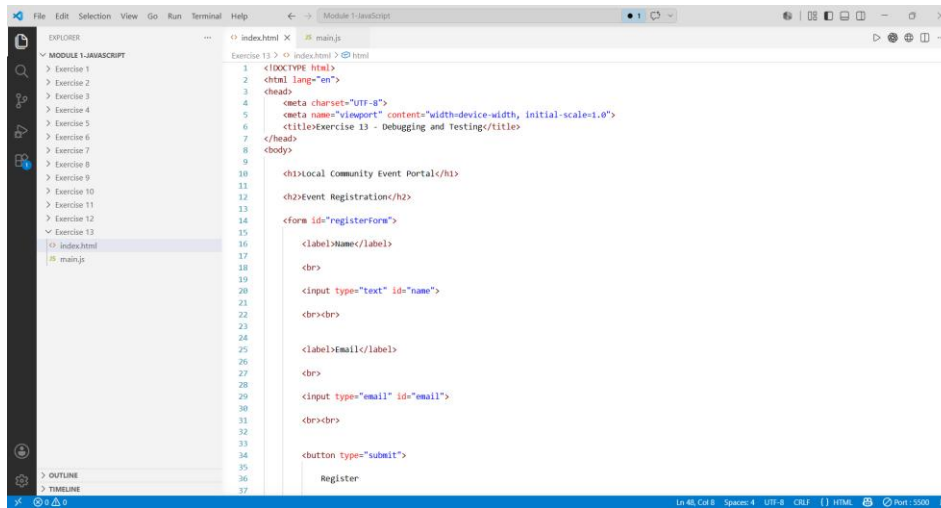
13. Debugging and Testing

Scenario: Registration is failing silently. You need to debug.

Objective: Use browser tools to inspect and fix issues.

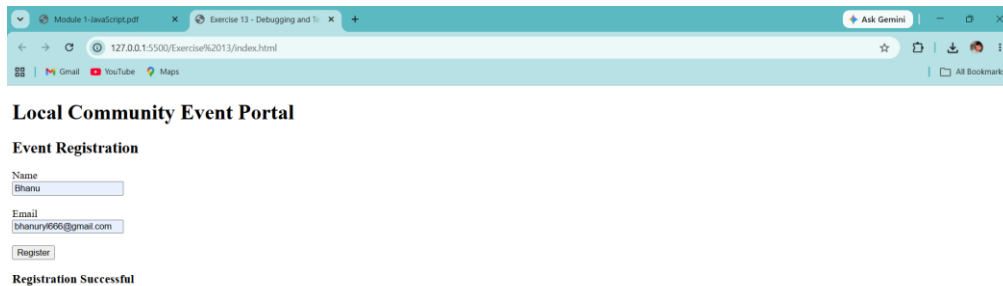
Task:

- Use Chrome Dev Tools Console and Network tab
- Add breakpoints and inspect variables
- Log form submission steps and check fetch request payload



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 13 - Debugging and Testing</title>
7 </head>
8 <body>
9
10  <h1>Local Community Event Portal</h1>
11
12  <h2>Event Registration</h2>
13
14  <form id="registerForm">
15
16    <label>Name</label>
17
18    <br>
19
20    <input type="text" id="name">
21
22    <br><br>
23
24    <label>Email</label>
25
26    <br>
27
28    <input type="email" id="email">
29
30    <br><br>
31
32    <button type="submit">
33      Register
34    </button>
35  </form>
36
37
```

Output:



Local Community Event Portal

Event Registration

Name
Bhanu

Email
bhanu0606@gmail.com

Register

Registration Successful

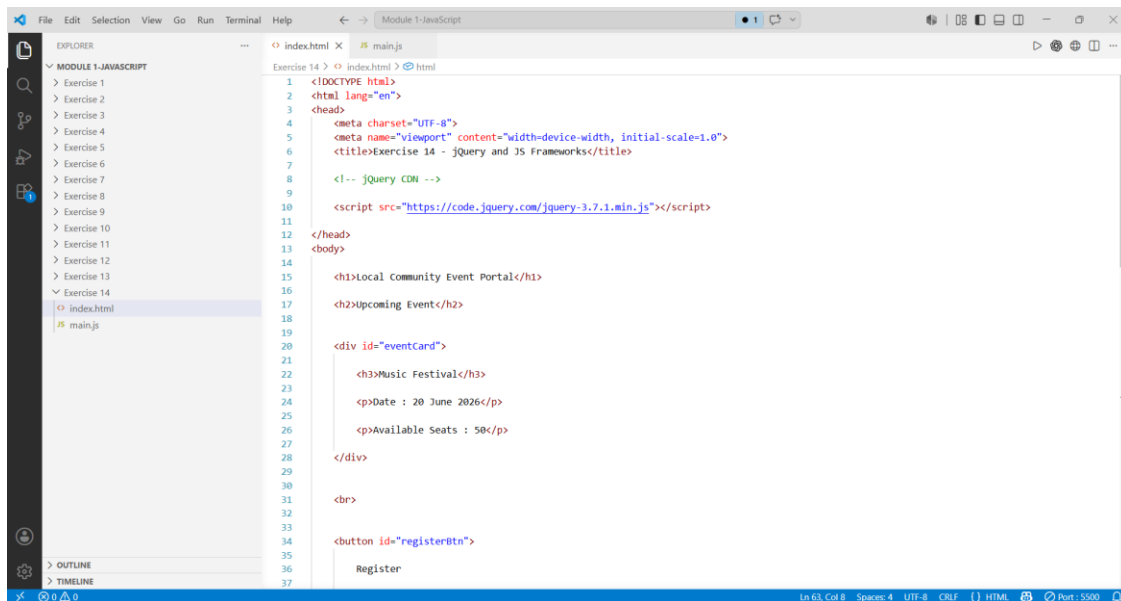
14. jQuery and JS Frameworks

Scenario: Use jQuery to simplify DOM tasks.

Objective: Understand and use jQuery.

Task:

- Use `$('#registerBtn').click(...)` to handle click events
- Use `.fadeIn()` and `.fadeOut()` for event cards
- Mention one benefit of moving to frameworks like React or Vue



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Exercise 14 - jQuery and JS Frameworks</title>
7
8   <!-- jQuery CDN -->
9
10  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
11
12 </head>
13 <body>
14
15   <h1>Local Community Event Portal</h1>
16
17   <h2>Upcoming Event</h2>
18
19   <div id="eventCard">
20
21     <h3>Music Festival</h3>
22
23     <p>Date : 20 June 2026</p>
24
25     <p>Available Seats : 50</p>
26
27   </div>
28
29   <br>
30
31   <button id="registerBtn">
32
33     Register
34
35
36
37
```

Output:



Local Community Event Portal

Upcoming Event

Music Festival

Date : 20 June 2026

Available Seats : 50

[Register](#) [Hide Event](#) [Show Event](#)

Framework Benefit

React and Vue help build reusable components and make UI updates easier.